

Process × Virtual Memory

进程身份怎样绑定地址空间，并在 `fork()` 后通过写时复制延迟分化

408 · OS-B02

Mother Interface

Process Identity → Address-Space Association

`fork`

→ Parent/Child Mapping Relation

`private write`

→ Protection Fault / COW Decision

→ Mapping Divergence if needed → Retry Write

本 Bridge 专门负责连接“谁在执行 (Process/Task)”和“它看到哪套地址空间 (Address Space / Page Table)”；不重新定义进程调度策略 (OS-01/02) 或硬件 MMU 地址翻译细节 (CO-07 / X-B02)。

先定义接口两侧的词

- **Process Identity (进程身份)**：OS 可独立命名、管理生命周期并关联资源环境的运行实例。Linux 的 `task_struct` 等结构只是工程表示，不是跨系统定义。
- **Address Space (地址空间)**：某运行实例可见的虚拟地址集合、区域权限、后备对象与地址翻译关系。Linux 常用 `mm_struct/VMA` 表示这些状态，但具体数据结构可变。
- **Page-table Selection State (页表选择状态)**：CPU 当前使用哪套地址翻译上下文。x86 CR3、RISC-V `satp`、ARM `translation table base` 是不同 ISA 的实例；同一地址空间内切换线程时未必改变这一状态。
- **Private Mapping Semantics (私有映射语义)**：`fork()` 后父子逻辑上应能独立修改的区域。实现可以先共享底层物理页，再把“第一次写”作为真正分化时点。
- **COW (Copy-on-Write, 写时复制)**：用共享物理内容 + 写保护/等价元数据延迟私有副本的创建。只有要求“以后写入彼此不可见”的映射才需要这类分化；显式共享映射不应被机械套成 COW。

Owners 与职责边界

- **左侧 Owner (OS-01/02 进程与调度)**：拥有进程生命周期、`fork()/exec()` 系统调用、进程上下文切换与调度决策。
- **右侧 Owner (OS-04 虚拟内存管理)**：拥有页表结构、物理页框 (Page Frame) 分配、缺页异常处理、页面置换策略。
- **本 Bridge Owns**：进程与地址空间的绑定关系、`fork()` 时页表项复制与写保护标记、写异常时的 COW 分化协议。
- **Stops**：硬件 MMU 逐级 Page Walk 与 TLB 命中判定归入跨科桥梁 X-B02。

1 核心机制：执行实体与地址空间是两条相关但不等价的关系

调度器切换执行实体时，先问两件事而不是默认“换进程就一定换页表”：

1. **Execution Context 是否变化？** 若 CPU 从一个可恢复执行实体切到另一个，需要保存/恢复其 PC、寄存器、SP 等现场；
2. **Address Space 是否变化？** 若新旧执行实体关联的是不同地址空间，CPU/内核必须把活动翻译上下文切到新地址空间；若只是同进程线程切换，地址空间可以保持不变。

因此稳定关系是：

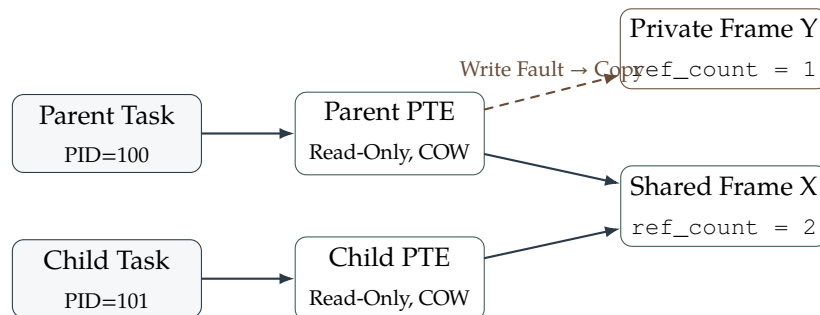
Execution Switch $\nRightarrow$ Address-Space Switch

而跨不同地址空间的切换必须保证后续地址翻译不会错误复用旧空间的映射。

1.1 ASID / PCID 硬件加速

地址空间切换要求 TLB 中旧翻译不能被错误用于新空间，但不等于“每次都必须整体清空 TLB”。现代体系结构可用 **ASID / PCID** 一类地址空间标签让不同空间的翻译并存；没有相应标签或实现选择不保留时，才需要更强的失效动作。Bridge 只保留这条一致性要求，具体硬件行为转 X-B02。

2 母模型：fork() 写时复制从共享到分化拓扑



3 写时复制 (COW) 的稳定四步执行轨迹

当 fork() 后父子对某个私有可写映射采用 COW，随后一方第一次写入该页时，可以用四步复原，而不绑定某个内核的字段名：

1. **建立父子关系：** 子进程得到逻辑上独立的地址空间描述与映射关系；对适合 COW 的私有页，父子页表/映射可暂时继续引用同一物理内容，并通过写保护或等价元数据记录“写时需要分化”；
2. **共享读取：** 只读访问不破坏父子隔离语义，因此可以继续从共享物理页读取；共享并非“零成本”，但避免了创建阶段的立即整页复制；
3. **第一次私有写触发保护异常：** 写操作与当前只读/受保护映射冲突，硬件把控制权交给内核；VM Owner 判断这不是非法写，而是可修复的 COW 情形；
4. **修复并重试：** 若仍有多多个需要保持独立语义的引用，内核为写方准备私有页内容并更新其映射；若实现能证明原页已经不再需要共享，也可采用等价的复用优化。随后使该写方拥有合法可写映射，维护必要的 TLB 一致性，并重新执行原写操作。

这里的页大小、引用计数字段、COW 标记位放在哪里、何时可以原地复用都属于具体 VM 实现；Bridge 只要求私有写不能泄漏到另一进程，而创建阶段又允许延迟实际复制。

4 四种内存共享与复制模型全景对比

共享/复制模型	内存与页表组织	数据可见性与隔离	典型场景与性能
写时复制 (COW Fork)	父子地址空间逻辑独立，私有页可暂时共享底层内容，写时按需分化	读取可共享；私有写后各自看到自己的版本	降低 fork() 创建阶段的复制成本，尤其适合随后很快 exec() 的路径
及早全量复制 (Eager Copy)	创建阶段即为需独立的数据建立物理副本	创建完成后无需再靠首次写触发分化	把成本前置；是否划算取决于实际会被修改/保留的数据量
显式共享映射 (Shared Mapping)	多个地址空间都建立到同一共享后备对象/页面的映射	按共享协议，写入应对其他映射者可见；仍需同步处理并发写	共享内存或 MAP_SHARED 一类 IPC/共享文件场景
同进程线程共享	多条执行流关联同一进程地址空间	硬件地址空间层面默认互相可寻址，不提供线程级内存隔离	省去跨地址空间切换，但共享可变状态需要 OS-03 的同步机制

Anti-Bridge：三个必须阻断的认知陷阱

- fork() 共享内存 ≠ IPC 共享内存：fork() 的共享是“只读暂存”，写操作发生后立刻分化；而 shmget/mmap (SHARED) 是持久双向可见的真正共享。
- 虚拟地址相同 ≠ 物理地址相同：父子进程中变量的虚拟地址（如 &val = 0x7ffd1200）完全相同，但在 COW 分化后，它们实际映射到了两个完全不同的物理页框！
- 进程切换 ≠ 重新拷贝内存：切换进程只需修改 CPU 的 CR3/satp 寄存器指针，物理内存中的数据原地不动。

30 秒极速调用协议

做题遇到“fork 后变量修改、进程内存隔离与共享”时，严格按三步判定：

私有映射初始可共享底层内容

写操作触发 COW 判断

必要时使写方映射分化并重试